

Examen réparti 2
Master 1 SAR
UE 4I401

L. Arantes, P. Cadinot, S. Dubois, J. Lejeune, J. Sopena, P. Sens

Janvier 2019

2 heures – Tout document papier autorisé
Tout appareil électronique interdit
Le barème sur 20 points est donné à titre indicatif

Exercice 1 – Questions de cours (1,5 points)

Question 1 $\frac{1}{2}$ point

Lors du traitement d'une interruption disque, est-il possible de traiter une interruption horloge ? Justifiez votre réponse.

Solution:

Oui, le niveau d'exécution du processeur autorise toutes les interruptions de niveau supérieur au disque (c'est le cas de l'horloge)

Question 2 $\frac{1}{2}$ point

Lorsqu'un signal est envoyé à un processus, quand, au plus tôt, ce processus traitera le signal et dans quel mode (utilisateur ou système) ?

Solution:

La prochaine fois qu'il est élu, lorsqu'il repasse en mode U

Question 3 $\frac{1}{2}$ point

Est-il possible qu'un processus soit à l'état prêt (SRUN) et reste non chargé en mémoire ? Si oui dans quel cas. Justifiez vos réponses.

Solution:

Oui, lorsque qu'un processus swappé à l'état de SSLEEP redevient prêt et qu'il n'y a plus suffisamment de processus à l'état SSLEEP ou SSTOP en mémoire.

Exercice 2 – Système de fichiers et Buffer Cache (6,5 points)

Les *inodes* ont une structure similaire à celle vue en TD et rappelée en annexe. La table des inodes sur disque commence au bloc 2. Les inodes ont une taille de 32 octets et leur numéro commence à partir de 1. Les blocs ont une taille de 512 octets.

Le répertoire racine correspond à l'inode numéro 20. Nous considérons les *inodes* et les blocs suivants :

inode 20	inode 30	bloc 201	bloc 402
i_number = 20	i_number = 30	.	1000
i_size	i_size = 5130	..	
i_addr[]	i_addr[]	fich	
0 100	0 302		
1 201	1 303		
.	.		
.	.		
9	9 503		
10	10 402		
11	11		
12	12		

Le bloc 100 ne contient pas la chaîne de caractère "fich".

Question 1 1/2 point

Quel est le numéro du dernier bloc de données du fichier "fich" ?

Solution:

Bloc numéro 1000

Question 2 1/2 point

Dans quels blocs sont stockés les inodes 20 et 30 ? Justifiez votre réponse.

Solution:

Les 2 inodes sont dans le bloc numéro 3

Justification : 16 inodes par bloc (512 div 32) Numéro = inumber div 16 | 1

Soit le code utilisateur suivant :

```

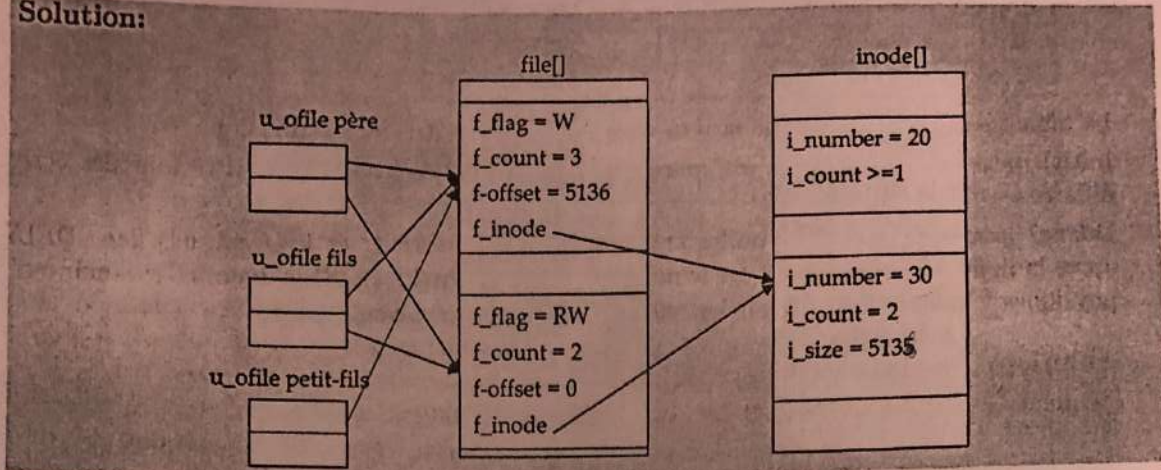
1  int f1, f2;
2  f1 = open("/fich", O_WRONLY | O_APPEND); /* O_APPEND
    positionne en fin de fichier */
3  f2 = open("/fich", O_RDWR);
4  if (fork() == 0) {
5      write(f1, "aaa", 3);
6      if (fork() == 0) {
7          close(f2);
8          write(f1, "bbb", 3);
9          exit(0);
10     }
11     wait(NULL);
12     exit(0);
13 }
14 wait(NULL);
15 close(f1); close(f2);
16 exit(0);
17 }
```


Don republi 2
2019- à approuver 1^{ère} feuille

Question 3 1½ points

Donnez l'état, juste après la ligne 8, des tables de descripteurs de fichiers, fichiers ouverts, et inodes (faites un schéma, pensez à représenter notamment les champs *f_count*, *f_offset*, *f_flags*, *i_count*, *i_size* et *i_number*).

Solution:



Question 4 2 points

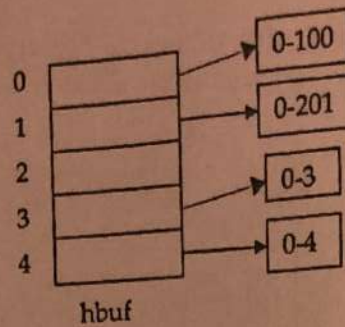
Avant l'exécution de ce code, nous faisons l'hypothèse que seule l'inode de la racine (inode numéro 20) est déjà présente dans la table `inode[]`. Donnez la séquence, dans l'ordre, des blocs et des inodes accédés jusqu'à la fin de la ligne 8. Pour chaque accès, spécifiez la ligne de code associée.

Solution:

2- open -namei
inode 20 en mémoire
accès bloc 100
accès bloc 201
accès bloc 3 (bloc de l'inode 30)
inode 30
3- open -namei
inode 20 en mémoire
accès bloc 100
accès bloc 201
accès inode 30 en mémoire
5- write
accès bloc 402 accès bloc 1000
8- write
idem ligne 5
séquence d'accès aux blocs : 100, 201, 3, 100, 201, 402, 1000, 402, 1000
séquence d'accès aux inodes : 20, 30, 20, 30

Question 5 2 points

Soit la configuration suivante du buffer cache avant l'exécution du code. Le buffer cache contient 4 blocs. Les numéros de bloc sont représentés sous la forme `devno-blockno`. La fonction de hachage associant un bloc à une des sous-listes est $(devno \mid blockno) \% 5$. Nous considérons que pour tous les blocs `devno = 0`.



La bfreelist est la suivante (de la tête vers la queue) : 0-201, 0-3, 0-100, 0-4.

Initialement aucun des blocs est marqué avec le flag DELWRI et les writes font des écritures différées.

Donnez la configuration du buffer cache ainsi que la bfreelist et les éventuels flags DELWRI après la ligne 8 du code. Quel est le nombre d'entrées/sorties (E/S) logiques et le nombre d'E/S physiques ? Vous indiquerez sur les quels blocs les E/S ont lieu.

Solution:

Séquence : 0-100, 0-201, 0-3, 0-100, 0-201, (0-402, 0-1000)x2

Accès : bfreelist

0-100 : 0-201, 0-3, 0-4, 0-100

0-201 : 0-3, 0-4, 0-100, 0-201

0-3 : 0-4, 0-100, 0-201, 0-3

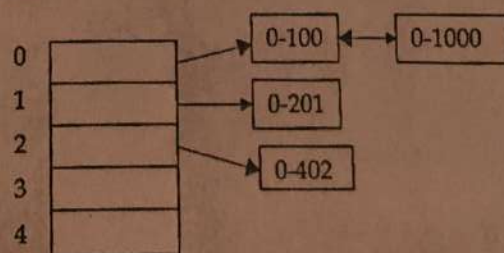
0-100 : 0-201, 0-3, 0-4, 0-100

0-201 : 0-3, 0-4, 0-100, 0-201

0-402 : 0-4, (1 E/S)

0-1000 : 0-100, 0-201, 0-402, 0-1000^{DELWRITE} (1 E/S)

2 E/S physique 9 E/S logiques



Exercice 3 – Swap (5 points)

Dans cette exercice, on considère que la mémoire comporte 20 unités et la table proc ne comporte que 7 processus. Voici l'état de cette table à T_0 juste après l'exécution de la fonction realtime qui a conduit à l'élection du processus 103. Dans cette table p_size est exprimé en nombre d'unités mémoire :

p_pid	p_uid	p_flag	p_stat	p_size	p_time	p_wchan	p_pri
0	0	SSYS SLOAD	SSLEEP	2	120/2	&runout	PSWP
1	0	SSYS SLOAD	SSLEEP	4	115/4	&proc[1]	PWAIT
101	1000	SLOAD	SSLEEP	1	1/1	&toto	50
102	1000	SLOCK SLOAD	SSLEEP	3	10/3	&toto	50
103	1000	SLOAD	SRUN	10	0/0		50
104	1000	SLOAD	SSLEEP	10	10/0	&toto	50
105	1000	0	SSLEEP	10	5/6	&toto	50

Pour simplifier le scénario, on fait les hypothèses suivantes :

- aucun fork ni exit, ni wakeup(&toto) ne sera appelé par la suite ;
- le temps d'exécution du code noyau (appels systèmes, swapper, init, ...) et les temps d'entrée/sortie sont considérés comme négligeables ;
- on ne prendra pas en compte les segments texte des processus.

Question 1 1 point

On suppose qu'à T_0 le processus 103 exécute l'instruction kill(104, SIGUSR1) puis entre dans une boucle infinie d'instructions ne comprenant pas d'appel système.

En vous appuyant sur les codes étudiés en TD, montrez comment l'exécution de cette instruction conduit à réveiller le swapper.

Solution:

L'appel système kill conduit à appeler psignal qui va faire un setrun car le processus 104 est endormi avec une priorité supérieur à 0. C'est cet appel à setrun qui va faire un wakeup(&runout)

On peut noter que les deux processus appartiennent à l'utilisateur d'uid 1000. Le processus 103 a donc le droit d'envoyer un message au processus 104.

Question 2 2 points

A quelle date le processus 104 va-t-il commencer à s'exécuter pour traiter le signal reçu ?

Pour justifier votre réponse vous indiquerez les actions du swapper depuis T_0 jusqu'à l'exécution du processus 104, en indiquant :

- la date
- le pid du processus concerné
- le sens du transfert : processus swappé ou processus ramené en mémoire

Solution:

T_0 : le swapper trouve le processus 101 qui est endormi et le swappe. Il vient de libérer 1 unité mémoire, ce qui n'est pas suffisant pour ramener 104. Il va donc s'endormir car 0, 1 et 102 sont non-swappable et 103 est en mémoire depuis moins de 2 secondes.

$T_0 + 1$: le swapper ne peut toujours rien faire

$T_0 + 2$: le swapper va placer 103 sur le disque car il est en mémoire depuis 2 secondes et que 104 est swappé depuis plus de 3 secondes. Il vient alors de libérer suffisamment de place pour ramener 104 en mémoire. Ce dernier va être élu car il est le seul processus à l'état SRUN en mémoire.

Question 3 ½ point
Sur quelle adresse le swapper est-il endormi lorsque le processus 104 commence à s'exécuter.
Vous justifierez votre réponse.

Solution:
Après avoir ramené en mémoire le processus 104, le swapper va s'endormir sur la variable `runin` car il vient de swapper le processus 103 qui est à l'état `SRUN`. Il va donc attendre qu'une place se libère en mémoire pour le ramener en mémoire.

Question 4 1½ points
En supposant que le signal `SIGUSR1` provoque un calcul de 9 secondes dans le processus 104 (le signal n'est pas ignoré et un handler correspondant à ce calcul a été enregistré correctement), combien de temps après l'envoi du signal (à T_0) va se terminer ce calcul ? Justifiez votre réponse.

Solution:
Seul les processus 103 et 104 sont alternativement swappables. Toutes les 3 secondes le swapper va donc inverser ces deux processus qui ont la même empreinte mémoire. On a donc 3 secondes d'exécution de 104 puis 3 secondes d'exécution de 103, et ainsi de suite. Le processus 104 terminera donc ses 9 secondes de calculs à la date $T_0 + 17 = T_0 + 2 + 3 + (3 + 3) + (3 + 3)$.
On peut noter qu'il n'y a pas de problème d'élection car un processus swappé ne peut pas être élu. C'est donc le processus qui a été ramené en mémoire qui s'exécutera.

Exercice 4 – Appel système - Fichiers (7 points)

Question 1 2 points

On souhaite écrire un nouvel appel système `int check_open(int idev, int inumber)`. Cet appel prend en entrée un numéro de partition `idev` et un numéro d'inode `inumber`. La fonction retourne 0, si l'inode de la partition n'est pas en cours d'utilisation par un processus et 1 sinon. Donnez le code dans le noyau de l'appel système `_check_open()` correspondant à `int check_open(int dev, int inumber)`.

Solution:

```
_check_open ( ) {  
    int idev = u.u_arg[0];  
    int inumber = u.u_arg[1];  
    struct inode *pi;  
    for (pi=&inode[0]; pi<&inode[NINODE]; pi++)  
        if (pi->i_dev == idev && pi->i_number == inumber && pi->i_count > 0) {  
            u.u_arg[R0] = 0;  
            return;  
        }  
    u.u_arg[R0] = 1;  
    return;  
}
```

Barème : boucle : 33%, test : 33%, paramètres : 33%, + 1% si 3 corrects

Question 2 2 points

On souhaite modifier l'appel système pour retourner 0 si l'inode n'est pas utilisée, 1 si elle est utilisée que par des processus en mode lecture uniquement et 2 si elle utilisée dans d'autres modes. Donnez le nouveau code de `_check_open()`.

Solution:

```
_check_open ( ) {
    int idev = u.u_arg[0];
    int inumber = u.u_arg[1];
    struct inode *pi;
    struct file *pf;
    for (pi=&inode[0]; pi<&inode[NINODE]; pi++)
        if (pi->i_dev == idev && pi->i_number == inumber && pi->i_count > 0)
            break;
    if (pi == &inode[NINODE]) {
        u.u_ar0[R0] = 0;
        return;
    }

    for (pf=&file[0]; pf<&file[NFILE]; pf++)
        if (pf->f_inode == pi && pf->f_flag&(FWRITE | FPIPE) ) {
            u.u_ar0[R0] = 2;
            return;
        }
    u.u_ar0[R0] = 1;
    return;
}
```

Question 3 3 points

On veut maintenant modifier l'appel système pour que `_check_open()` se bloque à la priorité `PWOPEN` si moins de 3 processus ont précédemment ouvert le fichier en lecture. `_check_open()` doit retourner 0 si le fichier n'est pas utilisé et 1 s'il est utilisé par au moins 3 processus en lecture. Modifiez la fonction `_check_open()` en conséquence et indiquez quelles sont les modifications à appliquer à `iget`, donnée en annexe, pour réveiller le processus.

Solution:

```
_check_open ( ) {
    int idev = u.u_arg[0];
    int inumber = u.u_arg[1];
    struct inode *pi;
    struct file *pf;
    int nb=0;

    for (pi=&inode[0]; pi<&inode[NINODE]; pi++)
        if (pi->i_dev == idev && pi->i_number == inumber && pi->i_count > 0)
            break;
    if (pi == &inode[NINODE]) {
        u.u_ar0[R0] = 0;
        return;
    }

    /* Modification :*/
    for (pf=&file[0]; pf<&file[NFILE]; pf++)
        if (pf->f_inode == pi && pf->f_flag&FREAD )
            nb++;

    if (nb < 3)
        sleep(pi+3, PWOPEN);

    u.u_ar0[R0] = 1;
    return;
}
```